

UDK Detection System — Project Report

2026-09-21 · @Someone

Overview

This is a real-time audio safety prototype that listens to a live audio stream, transcribes and analyzes it for **User Distress Keywords (UDKs)** — phrases like "call the police" or "someone is trying to hurt me" — and issues an escalating alert (`NO_ACTION` / `TRIGGER_VERIFY` / `TRIGGER_ALL`) when one is detected with enough confidence.

It started from a production-readiness review of a proposed audio-safety product brief, which found the brief's assumptions unsafe as specified: it treated "STT then keyword match" as sufficient detection (a single point of failure), treated network connectivity and audio storage as free, and had no answer for the network dropping mid-journey — the single most likely real-world failure mode for a personal-safety product. The corrected architecture built from that review is what this report covers.

Architecture

```
flowchart LR
    A[Live audio] --> B[VAD\nspeech segmentation]
    B --> C[STT\nspeech-to-text]
    B --> D[KWS\nkeyword spotting]
    C --> E[Matching engine\nexact + fuzzy + semantic]
    E --> F[Decision engine]
    D --> F
    F --> G[NO_ACTION / TRIGGER_VERIFY / TRIGGER_ALL]
```

Two independent detection paths feed the same decision engine: **STT + text matching** (three layers — exact substring, fuzzy/ASR-noise-tolerant, and semantic/paraphrase-tolerant) and a parallel **KWS (keyword spotting)** path that matches audio directly to enrolled reference phrases without needing transcription at all. If STT fails outright (an outage, or a bad transcription), KWS can still fire on its own — this is the actual single-point-of-failure removal the original review called for, built and verified, not just assumed.

Each UDK phrase carries a `verify_by_default` flag: a phrase common enough in ordinary conversation ("I don't feel safe here") only escalates to a full alarm after repetition, a second distinct UDK firing in the same window, or independent STT+KWS agreement — a

single hit lands at `TRIGGER_VERIFY` (needs confirmation) instead.

The system supports **five languages** — English, Hindi, Telugu, Kannada, Tamil — each with its own calibrated STT model, semantic matcher, and KWS backend, covered in detail in the Multilingual Support section.

Milestones M1–M7

All seven milestones from the roadmap are built and tested, each with a real bug found and fixed along the way — not just a feature checklist.

Milestone	What it built	Real bug found + fixed	Tests
M1	Core detection: exact/fuzzy/semantic matching, the decision contract (<code>NO_ACTION</code> / <code>TRIGGER_VERIFY</code> / <code>TRIGGER_ALL</code>), repetition + distinct-UDK escalation	—	10
M2	Personal UDK enrollment, chunked audio storage with per-segment checksums and gap detection	—	13
M3	Streaming WebSocket API, REST journey lifecycle, sequencing/acks/heartbeat, out-of-order frame buffering	Tail-segment finalization could stall forever on a one-shot utterance	24
M4	Failure injection: duplicate frames, crash recovery, full network loss/reconnect, delivery retries	Duplicate-frame handling bug in <code>AudioStore.write_segment</code>	20
M5	Durable DB (full event schema), concurrent-journey load test	SQLite multi-thread safety issue — <code>check_same_thread=False</code> does not make a <code>Connection</code> safe for concurrent use; fixed with an explicit <code>threading.Lock</code>	15
M6	Security/threat-model hardening: auth, rate limits, at-rest encryption, signed URLs, retention/deletion	A signed-URL token-parsing bug — a float's own <code>.</code> collided with the token's field separator	27
M7	Metrics/SLOs, concurrency soak test (60 concurrent journeys)	<code>JourneyStore</code> never evicted finished journeys from memory — a real long-running-process leak, fixed with <code>evict_finished()</code>	16

Beyond the roadmap

The original review deferred keyword spotting (KWS) and real semantic matching as v1 scope cuts (no training data existed for KWS; semantic matching started as a `rapidfuzz` stand-in). Both were revisited and built once time allowed:

- **Real semantic matching** (`semantic.py`): replaced the fuzzy-matching stand-in with a real pretrained sentence-embedding model for paraphrase detection ("he's going to hurt me" ↔ "someone is threatening me").
- **KWS** (`kws.py`): dataset-free keyword spotting via query-by-example — pretrained wav2vec2 audio embeddings + DTW (dynamic time warping) over frame-level features. Two CLAP-based formulations were tried first and rejected on real measured evidence (thin margins, wrong ordering) before landing on wav2vec2+DTW, which gave real, usable separation (same phrase: 0.22 DTW distance; different phrase: 0.27; unrelated: 0.36).
- Wiring KWS in surfaced a real bug: both `pipeline.py` and `api.py` skipped calling the decision engine entirely on an empty STT transcript — which would have silently defeated KWS-only detection during exactly the STT-outage scenario it exists for. Fixed to only skip when **both** signals are empty.

Full regression suite as of this report: **183/183 checks passing** across 11 test files (`test_pipeline.py` through `test_multilingual.py`), counted directly from each file's `[PASS]` lines.

Multilingual support: Hindi, Telugu, Kannada, Tamil

The recurring, hard-won lesson across every component below: **a model's "multilingual" label means broad pretraining, not per-language task-specific quality**. Every generic multilingual model tried first underperformed, sometimes catastrophically, and had to be replaced with a model actually fine-tuned for that specific language and task. This happened independently for STT, semantic matching, and KWS — three separate times, not a one-off.

STT: generic multilingual → language-specific fine-tune

Language	Best generic multilingual model	Language-specific fine-tune
Telugu	5% recall (<code>medium</code> , transcribed into the wrong script entirely — Devanagari, not Telugu)	100% recall
Kannada	45% recall (<code>medium</code> , correct script but too phonetically garbled to match)	95% recall
Hindi	90% recall (<code>small</code> ; <code>tiny</code> scored 0%, romanized/wrong-script output)	95% recall
Tamil	90% recall (<code>small</code>)	100% recall

The real-time latency crisis

The first fine-tuned checkpoints were plain `transformers` PyTorch models — measured at **37.29 seconds per 2-second clip**, completely non-viable for a system meant to alarm "within seconds." This was found only because latency was explicitly measured, not assumed from accuracy numbers alone. See the next section for the full fix.

Semantic matching: same lesson, second time

The generic multilingual embedding model used for English (`paraphrase-multilingual-MiniLM-L12-v2`) was tested against real Telugu/Kannada/Tamil sentence pairs and found **near-useless**: completely unrelated sentences ("I am hungry" vs. "someone is trying to hurt me") scored 0.56–0.89 — all clearing the 0.55 match threshold. Replaced with `LaBSE` (trained explicitly for cross-lingual retrieval across 109 languages) at a separately-calibrated threshold of 0.80, which gave clean, consistent separation across all four languages (100% recall / 0.5% FPR on the calibration corpus).

KWS: the same lesson, a third time

A raw self-supervised multilingual `wav2vec2` model (`wav2vec2-large-xlsr-53`) gave **zero usable signal**: 0/10 real paraphrases matched their own enrolled phrase as closest, with positive and negative distances completely overlapping. Root cause: this DTW-based approach needs a model fine-tuned for ASR (which sharpens word/phrase structure in its

hidden states), not just self-supervised pretraining — confirmed because English's own KWS model already was ASR-fine-tuned, which turned out to be *why* it worked. Fixed by switching to the Vakyansh project's per-language ASR-fine-tuned wav2vec2 models:

Language	Correct-closest	Threshold	Recall	FPR
Telugu	8/10	0.25	80%	15%
Kannada	7/10	0.20	60%	30%
Hindi	8/10	0.20	90%	10%
Tamil	7/10	0.25	70%	25%

Hindi's KWS quality is close to English's own; Kannada is consistently the weakest language across every component in this project.

Wired into the live API, not just proven in scripts

`api.py`'s `create_or_get()` takes per-language override maps (`stt_by_language` , `kws_by_language`) tried before falling back to the default shared backend — the same pattern used for the multilingual semantic matcher. Each is gated behind its own opt-in environment variable (`UDK_ENABLE_INDIC_STT` , `UDK_ENABLE_INDIC_KWS`) since each has a real one-time download/conversion cost. A journey's language is persisted through crash recovery, so a Hindi-speaking user's journey doesn't silently revert to English on restart.

The real-time latency investigation and fix

The core lesson: accuracy alone is not "solved" for a real-time system — latency needs its own separate measurement, and it nearly shipped unmeasured. The fine-tuned Indic STT models had excellent accuracy (95–100%) but were never timed until explicitly asked — at which point they measured **37.29 seconds per 2-second clip**, because they ran as plain `transformers` PyTorch inference on CPU, using an unmeasured "medium" model size chosen as a first guess.

Root cause and fix

Two independent problems, both real and both fixed:

1. **Wrong runtime.** Converting the same checkpoints to CTranslate2 format (`ctranslate2.converters.TransformersConverter`, int8 quantized) and running them through `faster-whisper` instead of raw `transformers.generate()` cut latency dramatically with zero accuracy change.
2. **Wrong model size, never measured.** "medium" was the original guess. Testing every size per language found smaller variants tied or beat it on accuracy while being far faster:

Language	Size tested	Recall	Latency
Telugu	medium	100%	14.41s (37.29s unconverted)
Telugu	small	100%	5.97s
Telugu	base (chosen)	100%	1.90s
Kannada	small	90%	6.99s
Kannada	base (chosen)	95%	2.33s
Hindi	small (chosen, no base exists)	95%	5.99s
Tamil	small (chosen, no base exists)	100%	6.81s

A further free optimization — `beam_size=1` (greedy decoding) instead of the default 5 — was tested and found zero accuracy cost across all four languages, for another 7–26% latency cut:

Language	beam_size=5	beam_size=1	Recall (both)
Telugu	2.53s	1.88s	100%
Kannada	1.57s	1.46s	95%
Hindi	3.91s	3.60s	95%
Tamil	5.59s	4.50s	100%

What this means for "is this actually real-time"

Telugu and Kannada now sit at ~1.5–2 seconds per segment — comparable to English's own SLO target and genuinely usable. Hindi and Tamil sit at ~3.6–4.5 seconds — usable, but not yet at English's level, because neither has a smaller fine-tuned variant available

from this model family. Stated honestly, not smoothed over: **three of five languages are demonstrably real-time-fast; two are "usable but slower,"** and closing that last gap needs either a smaller Hindi/Tamil fine-tune (none found yet) or accepting the current figure.

Two matching-layer bugs found and fixed

Bug 1: the fuzzy-layer false-positive gap (Telugu/Kannada/Hindi/Tamil)

Degraded-condition testing found Telugu/Kannada/Tamil's false-positive rate running well above English's (25–35% vs. 15%). Real diagnosis — translating ordinary negative sentences, synthesizing them, and tracing exactly which matching layer fired — found the false positives were overwhelmingly the **fuzzy layer**, not semantic (for Tamil, 5 of 6 false positives were fuzzy-layer). A threshold sweep on real audio found the fuzzy layer's raw match gate (0.55, tuned once for English and never re-checked per language) let false positives (0.65–0.81 raw similarity) and genuine matches (0.70–1.0) overlap badly — and this applied to **all four non-English languages, including Hindi**, whose problem had simply been masked by an earlier, different test corpus.

Fix: the fuzzy layer's raw gate became per-matcher (English keeps 0.55, unchanged; the shared multilingual matcher now uses 0.80). Result:

Language	FPR @ old gate (0.55)	FPR @ new gate (0.80)
Hindi	60%	0%
Telugu	60%	0%
Kannada	40%	0%
Tamil	65%	5%

Recall cost: negligible — any case that stopped clearing the fuzzy gate was still caught by the semantic layer's own 0.80 gate, since it was the same phrase with minor transcription noise, not a real paraphrase.

Bug 2: the generic-fragment false-alarm bug (all five languages)

Investigating *why* Hindi/Tamil read weaker than Telugu on the hardest audio conditions (heavy pitch/tempo distortion, overlapping speech) surfaced a more serious bug than a plain accuracy gap. Under heavy distortion, Hindi's and Kannada's speech-to-text sometimes collapsed to a single bare, generic word (e.g., Hindi “कॉन्फ़िडेंस टेस्ट टेस्ट” = “me”) instead of a garbled sentence. The exact-match layer's substring check had **no length or uniqueness guard**, so this meaningless fragment scored full confidence (1.0) — firing an **unconfirmed full alarm**, bypassing the “common phrase, verify first” safeguard entirely, on whichever UDK happened to contain it first in the list (usually the wrong one).

Checking the corpus confirmed this wasn't Hindi/Kannada-specific: it is a **structural bug present in every language's UDK list, including English** — “me” is a literal substring of 10 of English's own 20 UDK phrases, “help” of 5. It simply hadn't surfaced yet because English's STT hadn't been observed collapsing that far. The fuzzy layer had the identical vulnerability independently (its scoring method rewards a short fragment matching *somewhere* inside a longer phrase, with no penalty for how little of the phrase it actually covers).

Fix: a fragment that is a substring of **more than one** UDK phrase is now capped at a moderate confidence (0.65) in both layers — it still registers as something worth a confirmation prompt, but can no longer skip straight to an unconfirmed full alarm. A fragment unique to exactly one UDK keeps full confidence, since corpus-relative uniqueness is real evidence regardless of length. Verified directly against the real transcripts that exposed the bug: every previously-buggy case now lands at the safer, confirmation-required tier instead of a false full alarm, and every fragment that legitimately kept full confidence was independently confirmed correct. English's full corpus was re-run afterward and came back **byte-for-byte identical** (100/100/100/45/50% recall, 15%/10% FPR) — zero regression.

Checked whether KWS shares either bug — it doesn't

The generic-fragment bug was also tested against the audio-embedding KWS layer, using the exact fragment that broke the text layer. It failed to produce a false match at all: DTW distance came back at roughly 2–2.7× the match threshold, cleanly rejected. Root cause of the difference: DTW's distance is normalized over the *entire* alignment path, so a short query aligned against a long reference accumulates real cost for the reference's unmatched content — unlike text substring/fuzzy matching, which has no such penalty. This vulnerability is specific to the text-matching layers' scoring mechanics, not something the audio-embedding approach inherits.

Current status

183/183 regression checks passing, zero known code defects. What's left is real model/audio capability ceilings and human-review steps — not bugs.

Real, measured limitations

- **Hardest audio conditions remain genuinely hard.** Pitch/tempo-distorted and overlapping-speech recall sits at 30–50% across all five languages — Telugu tracks English almost exactly (both ~45–50%), suggesting the difficulty is inherent to the distortion technique itself, not language-specific. Hindi and Tamil read measurably worse than Telugu on these two conditions specifically; investigating this found and fixed the generic-fragment bug above, but the full gap isn't completely root-caused — the recall percentage itself was found to be noisy run-to-run (a TTS engine's stochastic synthesis, not a code issue), so a clean measurement needs a larger, fixed audio corpus not yet built.
- **Kannada is the most fragile language overall** — weakest on muffled/noisy audio, weakest KWS quality (60% recall/30% FPR), consistent with needing the most iteration throughout this whole project.
- **Hindi and Tamil are 2–3× slower than Telugu/Kannada** (3.6–4.5s vs. 1.5–2s per segment) — no smaller fine-tuned STT model exists for those two languages from this model family.
- **Tamil's FPR isn't fully at zero** even after the fuzzy-gate fix (5%, vs. 0% for the other three non-English languages).

Fallback coverage gap

`UDK_ENABLE_SEPARATION` (speech-separation retry for overlapping speech, real-measured to recover 15 recall points on English's hardest condition) and

`UDK_ENABLE_TIERED_STT` (confidence-gated STT fallback) exist **only for English** — never built or calibrated for the other four languages, despite overlapping speech being one of the two weakest conditions for Hindi/Tamil specifically.

Human-review gaps — no amount of testing substitutes for these

- **Translations were never reviewed by a native speaker.** Back-translation QA (round-tripping through a machine translation model) was done instead — 4 of 80 phrases were flagged, all investigated and found to be translation-tool artifacts, not real errors. This is an honest substitute, explicitly not a replacement.

- **The UDK phrase list itself has never been reviewed by a safety team**, in any language including English — the standing requirement that “the safety team owns the final list” hasn’t been exercised yet.
- **verify_by_default flags for the four non-English languages were copied 1:1 from English’s assignment** as a starting assumption, never independently re-measured against a real per-language false-positive corpus the way English’s own flags were (which took two real revisions to get right).
- **KWS calibration for the four non-English languages used a much smaller corpus** (10 positives + 20 negatives) than English’s (80 clips) — a real, evidence-based starting point, not a finished calibration.

Infrastructure

Postgres/Kafka/S3/KMS all have local stand-ins (SQLite, filesystem, a file-per-event queue) with matching interfaces, by design, for prototype speed — swapping them for the real services is a real, not-yet-done step before production, not a configuration flag.

Appendix: consolidated results

Degraded-condition recall, all five languages (current, post-fix)

Condition	English	Telugu	Kannada	Hindi	Tamil
Baseline	100%	100%	90%	100%	100%
Muffled	100%	100%	85%	95%	100%
Noisy	100%	100%	90%	100%	100%
Pitch/tempo-distorted	45%	45%	50%	30%	30%
Overlapping speech	50%	50%	45%	30%	45%
FPR, baseline negatives (current)	15%	0%	0%	0%	5%

STT final configuration per language

Language	Model	Size	Latency/segment	Clean recall
English	<code>tiny.en</code> (faster-whisper)	—	sub-second	100%

Language	Model	Size	Latency/segment	Clean recall
Telugu	vasista22/whisper-telugu-base (CTranslate2, int8)	base	1.88s	100%
Kannada	vasista22/whisper-kannada-base (CTranslate2, int8)	base	1.46s	95%
Hindi	vasista22/whisper-hindi-small (CTranslate2, int8)	small	3.60s	95%
Tamil	vasista22/whisper-tamil-small (CTranslate2, int8)	small	4.50s	100%

KWS calibration per language

Language	Model	Recall	FPR	Calibration corpus size
English	wav2vec2-base-960h + DTW	93.9%	15.0%	80 clips (calibrated threshold sweep)
Telugu	Vakyansh wav2vec2-telugu-tem-100	80%	15%	10 positives + 20 negatives
Kannada	Vakyansh wav2vec2-kannada-knm-560	60%	30%	10 positives + 20 negatives
Hindi	Vakyansh wav2vec2-hindi-him-4200	90%	10%	10 positives + 20 negatives
Tamil	Vakyansh wav2vec2-tamil-tam-250	70%	25%	10 positives + 20 negatives

Semantic matching configuration

Language	Model	Threshold	Calibration result
English	paraphrase-multilingual-MiniLM-L12-v2	0.55	Real true-positive pairs 0.59–0.76, true-negative pairs 0.15–0.26 — clean separation
Hindi/Telugu/Kannada/Tamil	LaBSE	0.80	100% recall / 0.5% FPR on a 10-positive/200-negative-pair-per-language corpus

Test suite breakdown (183 total)

File	Checks
test_m2.py	13
test_m3.py	24
test_m4.py	20
test_m5.py	15
test_m6.py	27
test_m7.py	16
test_semantic.py	10
test_pipeline.py	10
test_separation.py	6
test_kws.py	19
test_multilingual.py	23
Total	183